

xv6-riscv HAL 代码详解

从 RISC-V 到 LoongArch 的硬件抽象层设计

xiao-yang-lab

2026 年 6 月 13 日

目录

1 项目框架总览

1.1 三明治架构

整个项目的设计遵循”三明治”模式：

1. 顶层 (**kernel/**)：内核通用代码，只调用 `hal_` 前缀接口，不知道当前运行在哪个架构上
2. 中层 (**hal/hal_*.h**)：统一接口声明，定义所有 `hal_*` 函数的签名
3. 底层 (**hal/riscv/**, **hal/loongarch/**)：每个架构的独立实现，提供 `hal_*` 函数的具体实现

表 1: 文件组织与职责

层	文件	职责
统一入口	hal/hal.h	ARCH 条件包含, 聚合所有子系统头文件
接口声明	hal/hal_arch.h hal/hal_vm.h hal/hal_intr.h hal/hal_timer.h hal/hal_console.h hal/hal_ctx.h	CPU 控制接口 (CSR 读写) 虚拟内存接口 (TLB 刷新) 中断控制器接口 定时器接口 控制台 I/O 接口 上下文切换接口 (ARCH 条件结构体)
RISC-V 实现	hal/riscv/arch.h hal/riscv/memlayout.h hal/riscv/hal_entry.S hal/riscv/hal_start.c hal/riscv/hal_swch.S hal/riscv/hal_tramp.S hal/riscv/hal_kvec.S hal/riscv/hal_plic.c hal/riscv/hal_uart.c hal/riscv/hal_virtio.c hal/riscv/kernel.ld	CSR 内联函数、PTE 宏 (Sv39, 3 级) QEMU virt 物理地址布局 启动入口 (mhartid → 设栈 → start) M 态初始化 (PMP, 委托, mret) 上下文切换 (14 寄存器) 用户态陷入/返回 内核态中断向量 PLIC 中断控制器 16550a UART 驱动 Virtio-blk MMIO 驱动 链接脚本 (入口 0x80000000)
LoongArch 实现	hal/loongarch/arch.h hal/loongarch/memlayout.h hal/loongarch/hal_entry.S hal/loongarch/hal_start.c hal/loongarch/hal_swch.S hal/loongarch/hal_tramp.S hal/loongarch/hal_kvec.S hal/loongarch/hal_tlbrefill.S hal/loongarch/hal_intr.c hal/loongarch/hal_uart.c hal/loongarch/hal_virtio.c hal/loongarch/kernel.ld hal/loongarch/user.ld	CSR 内联、PTE 宏 (4 级)、异常码转换、RISC-V 兼容层 QEMU virt 物理地址、flash/RAM 分离布局 启动入口 (CUID → 设栈 → start) flash→RAM 搬运、多核同步 上下文切换 (12 寄存器) uservec/userret (KSave0/1 机制) 内核态中断向量 (17 寄存器) TLB 重填处理器 (lddir/ldpte walk) EIOINTC 中断控制器 16550a UART (QEMU 兼容) RAM-disk 块设备 链接脚本 (flash/RAM 分离) 用户程序链接 (0x2000 起始)

1.2 ARCH 条件编译分布

整个项目中 `#ifdef ARCH_loongarch` 共出现于 7 个文件:

表 2: 所有 ARCH 条件编译位置

文件	行号	用途
hal/hal.h	11-18	条件包含 arch.h 和 memlayout.h
hal/hal_ctx.h	10,26	struct hal_context 12 字段 vs 14 字段
kernel/vm.c	34,62,117,355,556	kvmmake, MMU 使能, freewalk, tlb_fill
kernel/proc.c	188,217	跳过 TRAMPOLINE/TRAPFRAME 映射
kernel/exec.c	57-71	低 2 页 PLV0 保护页映射
kernel/trap.c	119-125	KSave1 存储陷阱帧内核 VA
Makefile	多处	工具链、QEMU 选项、用户链接

关键设计原则：只有 7 个文件需要 **ARCH** 条件编译，其余 20+ 个内核文件完全平台无关。

2 阅读源码的顺序

推荐按以下顺序阅读，从”宏观架构”到”微观实现”逐步深入：

1. **Makefile** (10 分钟)：理解 ARCH 切换机制、编译标志差异、QEMU 启动方式
2. **hal/hal.h + 7 个 hal/hal_*.h** (15 分钟)：理解 HAL 接口层定义了哪些统一 API
3. **hal/riscv/arch.h + hal/loongarch/arch.h** 对照读 (20 分钟)：重点是 PTE 格式、PT_LEVELS、MAXVA、estat_to_scause 这几个核心差异
4. **hal/riscv/memlayout.h + hal/loongarch/memlayout.h** 对照读 (10 分钟)：理解两个平台的物理地址布局
5. **kernel/vm.c** (30 分钟)：这是 HAL 化最深入的文件。重点看 walk()(跨架构)、kvmminithart() (两平台 MMU 使能)、vmfault() (惰性分配 + TLB 填充)
6. **hal/loongarch/hal_tlbrefill.S** (20 分钟)：LoongArch 最核心的文件。逐行理解 lddir 4 级 walk + P/W 清除 + MAXVA 检查
7. **hal/riscv/hal_tramp.S + hal/loongarch/hal_tramp.S** 对照读 (15 分钟)：对比 sscratch vs KSave0/1 两种 trapframe 访问机制
8. **kernel/trap.c** (15 分钟)：usertrap/devintr/prepare_return 的 HAL 迁移
9. **hal/riscv/hal_start.c + hal/loongarch/hal_start.c** 对照读 (10 分钟)：M 态初始化 vs flash→RAM 搬运
10. **hal/*/hal_swch.S** (5 分钟)：12 vs 14 寄存器的上下文切换
11. **hal/*/hal_virtio.c** (10 分钟)：MMIO virtio vs RAM-disk 两种块设备方案
12. **hal/*/hal_intr.c** (10 分钟)：PLIC vs EIOINTC 的 claim/complete 协议
13. 其余文件：hal_entry.S, hal_uart.c, hal_kvec.S, kernel/exec.c, kernel/proc.c, kernel/console.c, kernel/main.c (各 5-10 分钟)

总计约 3-4 小时可完成全部源码阅读。

3 文件之间的协同作用

3.1 启动流程中的文件协作

QEMU 加载内核

kernel.ld	决定入口地址和内存布局
hal_entry.S	第一条指令，设栈后跳 start()
hal_start.c	start() 初始化硬件
	[RISC-V: M态配置+mret→main]
	[LoongArch: flash→RAM搬运+多核同步]
kernel/main.c	main()
kinit()	kalloc.c 物理页分配器
kvminit()	vm.c 内核页表
kvminithart()	vm.c MMU使能(DMW/MMU)
procinit()	proc.c 进程表
trapinit()	trap.c 陷阱向量
trapinithart()	trap.c 每核陷阱设置
hal_irq_init()	hal_intr.c 中断控制器
hal_irq_hart_init()	
virtio_disk_init()	hal_virtio.c 块设备
userinit()	proc.c 第一个用户进程
scheduler()	proc.c 调度循环
hal_switch()	hal_swth.S 上下文切换

3.2 陷阱处理流程中的文件协作

用户态执行 → TLB miss / syscall / 中断

[LoongArch TLB miss]

hal_tlbrefill.S lddir/ldpte walk → tlbfill → ertn
 (若失败)
 .Ltlbr_invalid → 缺页异常 → uservec

[异常/中断进入内核]

hal_trampoline.S (uservec)
 保存寄存器 → 切内核PGDL → jirl usertrap

kernel/trap.c usertrap()

hal_read_scause() → arch.h:estat_to_scause()
 syscall() → syscall.c → sysfile.c/sysproc.c
 devintr() → hal_irq_claim() → hal_intr.c
 vmfault() → vm.c (惰性分配+TLB填充)

prepare_return() → hal_trampoline.S (userret)
 恢复寄存器 → 切用户PGDL → eretn

[内核态中断]

hal_kvec.S (kernelvec)
 保存→kerneltrap()→恢复→eretn

3.3 上下文切换的文件协作

scheduler() [proc.c]

hal_switch(&old->context, &new->context)

hal_swch.S
 保存old的callee-saved寄存器到struct hal_context
 从struct hal_context加载new的callee-saved寄存器
 jirl \$r0, \$ra, 0 → 跳转到new->context.ra (即forkret)

forkret() [proc.c]

[首次] fsinit() + kexec("/init")
 prepare_return() → userret → eretn → 用户态

3.4 页表管理的文件协作

vm.c: walk()	← 使用 arch.h 的 PT_LEVELS, PX(), PXSHIFT
vm.c: mappages()	← 使用 arch.h 的 PA2PTE, PTE_V_CACHE
vm.c: kvminithart()	← 使用 arch.h 的 w_dmw0, w_pgdl, w_crmd 使用 memlayout.h 的 KERNBASE, PHYSTOP
vm.c: vmfault()	← 调用 tlb_fill_from_pte() (LoongArch) 使用 arch.h 的 PTE_U, PTE_W
hal_tlbrefill.S	← 使用 arch.h 未显式引用但假定 PWCL/PWCH 已配 使用 CSR 0x19(PGDL), 0x89(TLBRBADV)

4 编译过程：每个文件如何被调用

4.1 ARCH=riscv 编译流程

make ARCH=riscv

```
CFLAGS += -march=rv64gc -mmodel=medany -DARCH_riscv
-Ihal -Ihal/riscv          ← arch.h → hal/riscv/arch.h
                           memlayout.h → hal/riscv/memlayout.h
```

编译 kernel/*.c ← 通用内核，调用 hal_*() 函数

编译 hal/riscv/hal_*.c hal_*.S ← RISC-V 平台实现

链接 kernel/kernel

```
-T hal/riscv/kernel.ld      ← 入口 0x80000000
kernel/*.o + hal/riscv/*.o
```

编译 user/*.c ← 用户程序 (RISC-V)

```
-T user/user.ld (or default)
```

qemu:

```
qemu-system-riscv64 -kernel kernel/kernel
-machine virt -m 128M -smp 3
-drive file=fs.img,if=none,format=raw,id=x0
-device virtio-blk-device,drive=x0
```

4.2 ARCH=loongarch 编译流程

make ARCH=loongarch

```
TOOLPREFIX = loongarch64-linux-gnu-
CFLAGS += -march=loongarch64 -mmodel=normal -DARCH_loongarch
-Ihal -Ihal/loongarch      ← arch.h → hal/loongarch/arch.h
                           memlayout.h → hal/loongarch/memlayout.h
```

编译 kernel/*.c ← 同一份通用代码！

条件编译：#ifdef ARCH_loongarch 生效

vm.c: PT_LEVELS=4, kvminithart 走 LoongArch 分支

proc.c: 跳过 TRAMPOLINE 映射

exec.c: 映射低2页保护页

trap.c: 写入 KSave1

编译 hal/loongarch/hal_*.c hal_*.S ← LoongArch 平台实现
hal_tlbrefill.S ← TLBENTRY 处理函数
hal_trampoline.S ← KSave0/1 版本
hal_virtio.c ← RAM-disk 实现

嵌入 fs.img:

```
objcopy -I binary -O elf64-loongarch fs.img kernel/fs_img.o
```

链接 kernel/kernel

```
-T hal/loongarch/kernel.ld          ← 入口 0x1C000000  
kernel/*.o + hal/loongarch/*.o + kernel/fs_img.o
```

```
objcopy -O binary kernel/kernel kernel/kernel.bin
```

编译 user/*.c ← 用户程序 (LoongArch)

```
-T hal/loongarch/user.ld          ← 从 0x2000 链接
```

usys.S: sed 转换 RISC-V 指令 → LoongArch

```
"li a7, N" → "ori $a7, $r0, N"  
"ecall"    → "syscall 0"  
"ret"      → "jirl $r0, $ra, 0"
```

qemu:

```
qemu-system-loongarch64 -bios kernel/kernel.bin  
-machine virt -cpu la464 -m 1G -smp 1  
-nographic -nic none
```


4.3 关键编译差异总结

表 3: RISC-V vs LoongArch 编译差异

项目	RISC-V	LoongArch
ARCH 宏	-DARCH_riscv	-DARCH_loongarch
march/mcmodel	rv64gc / medany	loongarch64 / normal
TOOLPREFIX	riscv64-unknown-elf-	loongarch64-linux-gnu-
链接脚本	hal/riscv/kernel.ld	hal/loongarch/kernel.ld
入口地址	0x80000000 (RAM)	0x1C000000 (flash)
PT_LEVELS	3	4
MAXVA	$1 \ll 38$	$1 \ll 38$ (保持 xv6 契约)
用户链接基址	0x0	0x2000 (跳过保护页)
QEMU 启动	-kernel (ELF)	-bios (raw binary)
文件系统	-drive virtio	objcopy 嵌入内核
HAL 对象	hal/riscv/*.o	hal/loongarch/*.o + hal_tlbrefill.o

5 RISC-V HAL: 哪些是直接移植的 xv6 源码

5.1 直接移植的文件 (修改量 < 10%)

以下文件几乎原封不动来自 xv6-riscv 原始源码, 只需要修改 `#include` 和函数命名:

表 4: 直接移植的 xv6 源码

文件	原始 xv6 来源	为什么可以直接移植
hal/riscv/arch.h	kernel/riscv.h	CSR 内联函数和 PTE 宏是 RISC-V 架构定义的, 与 xv6 无关。只需在末尾加 <code>hal_*</code> 包装器即可成为 HAL 实现
hal/riscv/hal_entry.S	kernel/entry.S	启动入口逻辑完全一致 (读 mhartid、设栈、跳 start)。只是文件位置变了
hal/riscv/hal_start.c	kernel/start.c	M 态初始化 (PMP、委托、mret) 是 RISC-V 特权架构规定流程, 不可变
hal/riscv/hal_swch.S	kernel/swch.S	14 寄存器的保存/恢复由 RISC-V ABI 定义, 不受 OS 设计影响
hal/riscv/hal_trampoline.S	kernel/trampoline.S	用户态陷入/返回的寄存器保存顺序、sscratch 技巧、sfence.vma 用法全部保留
hal/riscv/hal_kvec.S	kernel/kernelvec.S	内核态中断向量只保存 caller-saved 寄存器, 由 RISC-V ABI 定义
hal/riscv/hal_plic.c	kernel/plic.c	PLIC 是 SiFive 标准中断控制器, 寄存器布局公开且固定
hal/riscv/hal_uart.c	kernel/uart.c	16550a UART 是工业标准, 寄存器定义和初始化流程不改
hal/riscv/hal_virtio.c	kernel/virtio_disk.c	MMIO virtio-blk 协议是公开标准, QEMU 实现兼容
hal/riscv/kernel.ld	kernel/kernel.ld	链接脚本入口地址、段布局基本不变, 只增加 trampsec 对齐
hal/riscv/memlayout.h	kernel/memlayout.h	QEMU virt 机器的物理地址是固定的, 不会变化

移植方式: 基本上是”搬家 + 改 include + 加 `hal_*` 包装函数”。核心逻辑一行未改。

5.2 为什么可以直接移植

RISC-V HAL 可以直接移植原始 xv6 代码, 原因是:

1. **RISC-V** 是 **xv6** 的原生平台: xv6-riscv 就是为 RISC-V 写的, 所有 CSR 操作、异常处理、页表格式都是原生支持的
2. **HAL** 化不影响 **RISC-V**: HAL 的设计目标之一就是 RISC-V 作为第一个平台时”只搬家不改逻辑”。`hal_*` 包装器就是对原有 CSR 函数的重命名, 不改变行为
3. 硬件接口标准化: PLIC、16550a UART、MMIO virtio 都是工业标准, QEMU 的模拟与

真实硬件完全兼容

5.3 需要重写的部分

即使是 RISC-V HAL，也有少量需要新增的内容：

- `hal_*` 包装函数 (`arch.h` 末尾 19 个 `inline`)：这是 HAL 层新增的，原始 xv6 没有
- `hal/hal_*.h` 接口头文件：7 个统一接口声明文件，全部是 HAL 新增
- `hal/hal.h`：统一入口，聚合所有子系统
- `hal/hal_ctx.h`：将 `struct context` 改为 `struct hal_context`，支持 ARCH 条件编译（为 LoongArch 预留 12 字段版本）
- `kernel/vm.c` 的跨架构修改：`walk()` 用 `PT_LEVELS`、`mappages` 用 `PTE_V_CACHE`、`kvminithart` 增加 ARCH 分支

6 LoongArch HAL 代码逐行详解

以下对 LoongArch HAL 的 5 个最核心文件进行逐段代码级讲解。

6.1 `hal/loongarch/arch.h` 详细解析

6.1.1 CSR 寄存器映射（行 16-37）

LoongArch 的 CSR 没有类似 RISC-V 的符号名称，操作系统必须用裸编号访问。每个 CSR 定义一个 `#define` 宏，例如：

```

1 #define CSR_CRMD    0x0      // Current Mode: PLV[1:0], IE[2], DA[3], PG[4]
2 #define CSR_PRMD    0x1      // Previous Mode: PPLV[1:0], PIE[2]
3 #define CSR_ECFG    0x4      // Exception Config: LIE[12:0]
4 #define CSR_ESTAT    0x5      // Exception Status: IS[12:0], Ecode[21:16]
5 #define CSR_ERA      0x6      // Exception Return Address (= sepc)
6 #define CSR_BADV     0x7      // Bad Virtual Address (= stval)
7 #define CSR_EENTRY   0xC      // Exception Entry Point (= stvec)
8 #define CSR_PGDL     0x19     // Page Table Base (low half)
9 #define CSR_TLBRENTY 0x88     // TLB Refill Entry

```

设计要点：CSR 编号来自 LoongArch 手册 §7。这些编号是硬件定义的，不能随意更改。

6.1.2 CRMD/PRMD 位域定义（行 40-51）

```

1 #define CRMD_PLV    0x3      // PLV mask (bits 1:0)
2 #define CRMD_IE     (1L << 2) // global interrupt enable
3 #define CRMD_DA     (1L << 3) // direct address translation enable
4 #define CRMD_PG     (1L << 4) // page table mapping enable
5
6 #define PRMD_PPLV   0x3      // previous PLV
7 #define PRMD_PIE    (1L << 2) // previous interrupt enable
8 #define PRMD_USER   (3L)     // PPLV=3 → returning to user mode

```

CRMD 相当于 RISC-V 的 `mstatus+sstatus` 的组合：

- PLV[1:0]: 当前特权级 (0= 内核, 3= 用户)
- IE: 全局中断使能
- DA: 直接地址翻译 (DA=1 时 VA=PA, 绕过 MMU)
- PG: 页表映射使能 (DA=0, PG=1 时启用 MMU)

PRMD 在异常发生时由硬件自动保存 CRMD.PLV 和 CRMD.IE, ertn 时恢复。

6.1.3 ECFG/ESTAT 位域定义 (行 54-66)

```

1 #define ECFG_LIE_TIMER (1L << 11) // local timer interrupt enable
2 #define ECFG_LIE_HWI   (1L << 12) // local hardware interrupt enable
3
4 #define ESTAT_IS_TIMER (1L << 11) // timer interrupt pending
5 #define ESTAT_IS_HWI   (1L << 12) // external hardware interrupt pending
6 #define ESTAT_ECODE_SHIFT 16
7 #define ESTAT_ECODE_MASK (0x3FULL << ESTAT_ECODE_SHIFT)

```

ECFG 是中断使能寄存器, ESTAT 是中断状态 + 异常码寄存器。关键设计: ECFG 和 ESTAT 使用相同的位位置 (bit 11= 定时器, bit 12= 外部中断), 这样 ECFG.LIE 和 ESTAT.IS 可以直接做位与来判断中断源。

6.1.4 异常码定义 (行 70-87)

```

1 #define EXCCODE_INT      0 // interrupt
2 #define EXCCODE_PIL      1 // page invalid on load
3 #define EXCCODE_PIS      2 // page invalid on store
4 #define EXCCODE_PIF      3 // page invalid on instruction fetch
5 #define EXCCODE_PME      4 // page modification
6 #define EXCCODE_PNR      5 // page not readable
7 #define EXCCODE_PNX      6 // page not executable
8 #define EXCCODE_PPI      7 // page privilege violation
9 #define EXCCODE_ADE      8 // address error
10 #define EXCCODE_SYS     11 // system call
11 #define EXCCODE_INE     13 // instruction not exist
12 #define EXCCODE_IPE     14 // instruction privilege error

```

关键理解: 这些是 LoongArch 的异常码, 与 RISC-V scause 完全不同。例如 LoongArch 的 SYS=11 对应 RISC-V scause=8 (syscall)。这就是为什么需要 estat_to_scause() 做转换。

6.1.5 CSR 访问内联函数 (行 106-225)

每个 CSR 都有一对 read/write 函数, 例如 CRMD:

```

1 static inline uint64 r_crmd(void) {
2     uint64 x;
3     asm volatile("csrrd %0, 0x0" : "=r"(x));
4     return x;
5 }
6 static inline void w_crmd(uint64 x) {
7     asm volatile("cswr %0, 0x0" : : "r"(x));

```

8 }

csrrd 语义：读取 CSR 的值到通用寄存器。“=r”(x) 约束告诉编译器 x 是输出。**csrwr** 语义：将通用寄存器的值写入 CSR，同时将旧 CSR 值写回寄存器。“:r”(x) 约束表示 x 只作为输入，旧值被丢弃。

为什么用 **csrwr** 而不是 **csrxchg**：csrwr 是简单写，csrxchg 需要掩码寄存器。对于只需要设置某个值的场景，csrwr 更简洁。但如果需要修改 CSR 的特定位（如只改 CRMD.DA），应该用读-改-写模式（先 csrrd，修改，再 csrwr）。

6.1.6 中断控制（行 230-242）

```
1 static inline void intr_on(void) {
2     uint64 x = r_crmd();
3     w_crmd(x | CRMD_IE);
4 }
5 static inline void intr_off(void) {
6     uint64 x = r_crmd();
7     w_crmd(x & ~CRMD_IE);
8 }
9 static inline int intr_get(void) {
10     return (r_crmd() & CRMD_IE) != 0;
11 }
```

为什么用读-改-写而不是 **csrxchg**：csrxchg 的语义是 $CSR = (CSR \& \sim mask) | (val \& mask)$ ，需要额外的掩码计算。读-改-写模式更直观且不容易出错。

6.1.7 TLB 刷新（行 247）

```
1 static inline void sfence_vma(void) {
2     asm volatile("invtlb 0, $r0, $r0");
3 }
```

invtlb op=0 表示“清除所有 TLB 条目”。rj=\$r0 表示不筛选 ASID，rk=\$r0 表示不筛选 VA。这等价于 RISC-V 的 `sfence.vma zero, zero`。

6.1.8 GP 寄存器访问（行 255-275）

```
1 static inline uint64 r_tp(void) {
2     uint64 x;
3     asm volatile("or %0, $r2, $r0" : "=r"(x));
4     return x;
5 }
6 static inline void w_tp(uint64 x) {
7     asm volatile("or $r2, %0, $r0" : : "r"(x));
8 }
```

为什么用 `or $r2, %0, $r0`：LoongArch 没有 RISC-V 那样的 mv 伪指令。or rd, rj, \$r0 是“移动”的标准实现（OR 与零 = 恒等）。\$r2 是 LoongArch 的 tp（线程指针）寄存器。

6.1.9 estat_to_scause() 逐行解析 (行 280-313)

```

1 static inline uint64 estat_to_scause(uint64 estat) {
2     uint64 ecode = (estat >> ESTAT_ECODE_SHIFT) & 0x3F;           // 提取 Ecode[5:0]
3     uint64 is_bits = estat & 0x1FFF;                               // 提取 IS[12:0]
4     uint64 esubcode = (estat >> ESTAT_ESUBCODE_SHIFT) & 0x1FF;    // 提取 EsubCode[8:0]
5
6     // 中断分支: Ecode=0
7     if (ecode == EXCCODE_INT) {
8         if (is_bits & ESTAT_IS_TIMER)
9             return 0x8000000000000005ULL; // RISC-V supervisor timer interrupt
10        if (is_bits & ESTAT_IS_HWI)
11            return 0x8000000000000009ULL; // RISC-V supervisor external interrupt
12        return 0x8000000000000000ULL | ecode;
13    }
14
15    // 异常分支: 映射 LoongArch Ecode → RISC-V scause
16    switch (ecode) {
17        case EXCCODE_SYS: return 8; // 11 → 8 (syscall)
18        case EXCCODE_PIL: return 13; // 1 → 13 (load page fault)
19        case EXCCODE_PIS: return 15; // 2 → 15 (store page fault)
20        case EXCCODE_PIF: return 12; // 3 → 12 (instr page fault)
21        case EXCCODE_PME: return 15; // 4 → 15 (page mod → store fault)
22        case EXCCODE_PNR: return 13; // 5 → 13 (non-readable → load fault)
23        case EXCCODE_PNX: return 12; // 6 → 12 (non-exec → instr fault)
24        case EXCCODE_PPI: return 13; // 7 → 13 (privilege → load fault)
25        case EXCCODE_ADE: // 8 → avoid syscall conflict!
26            return esubcode == 0 ? 12 : 13; // ADEF→12, ADEM→13
27        case EXCCODE_INE: return 2; // 13 → 2 (illegal instruction)
28        case EXCCODE_IPE: return 12; // 14 → 12 (instr priv → instr fault)
29        case EXCCODE_BP: return 3; // 12 → 3 (breakpoint)
30        default: return 0x100 | ecode; // 未知: 加偏移避免冲突
31    }
32 }

```

逐条解析:

1. **ecode** 提取: ESTAT[21:16] 是 6 位异常码。右移 16 位后与 0x3F 做 AND。
2. **is_bits** 提取: ESTAT[12:0] 是中断挂起位图。与 0x1FFF 做 AND。
3. 中断分支: ecode=0 时, 根据 IS 位判断中断类型。bit 11= 定时器 → 0x8000000000000005, bit 12= 外部 → 0x8000000000000009。这两个值与 RISC-V 的 timer/external interrupt scause 值完全一致。
4. **PIL/PIS/PIF**: LoongArch 将缺页细分为 load(1)、store(2)、fetch(3) 三种, 分别映射到 RISC-V 的 13(load)、15(store)、12(instr)。
5. **PME**: 页修改异常 (向只读页写入)。LoongArch 将此作为独立异常 (4), 但 RISC-V 将其视为 store page fault(15)。统一映射为 15。
6. **ADE** 的特殊处理: LoongArch 的 ADE(8) 恰好与 RISC-V 的 syscall(8) 冲突! 必须根据 EsubCode 区分: ADEF(0) → 12(instr fault), ADEM(1) → 13(load fault)。
7. **IPE** 的修正: 指令特权异常 (14) 映射为 12(instr page fault) 而非 2(illegal instruction)。

因为 RISC-V 对指令特权错误的处理也是 page fault，而非 illegal instruction。

8. **default** 安全处理：未知异常码返回 0x100 | ecode。OR 0x100 确保返回值与任何已知的 RISC-V scause 值不冲突（所有 RISC-V scause 都小于 0x40），方便在 usertrap 中打印诊断信息。

6.1.10 RISC-V 兼容别名（行 378-410）

```

1 #define w_satp(x)    w_pgdl(x)    // satp → PGDL
2 #define r_satp()    r_pgdl()
3 #define w_stvec(x)  w_eentry(x)  // stvec → EENTRY
4 #define r_stvec()   r_eentry()
5 #define sfence_vma() asm volatile("invtlb 0, $r0, $r0")

```

这些 #define 是 HAL 最关键的设计决策之一。内核通用代码（vm.c, trap.c, proc.c）中仍然存在对 RISC-V CSR 函数名的直接调用。通过 #define 别名让这些调用在 LoongArch 编译时自动变为对应的 LoongArch CSR 操作，无需修改内核代码。

SSTATUS 兼容函数（行 382-410）：

```

1 static inline uint64 r_sstatus_compat(void) {
2     uint64 prmd = r_prmd();
3     uint64 sstatus = 0;
4     if ((prmd & 0x3) != 3) sstatus |= (1L << 8); // SPP=1 if from kernel
5     if (prmd & PRMD_PIE) sstatus |= (1L << 5); // SPIE=1 if IE was on
6     return sstatus;
7 }
8 static inline void w_sstatus_compat(uint64 x) {
9     uint64 prmd = 0;
10    if ((x & (1L << 8)) == 0) prmd |= 3; // SPP=0 → PPLV=3 (user)
11    if (x & (1L << 5)) prmd |= PRMD_PIE;
12    w_prmd(prmd);
13 }

```

r_sstatus_compat 的逻辑：

- 读 PRMD.PPLV（异常前的特权级）：如果是 3（用户态），返回 SPP=0；否则 SPP=1（内核态）
- 读 PRMD.PIE（异常前的中断使能）：如果为 1，返回 SPIE=1
- 构造一个“假的”RISC-V sstatus 值返回

w_sstatus_compat 的逻辑（反向）：

- 从 RISC-V sstatus 格式提取 SPP 位：SPP=0 → PPLV=3（即将返回用户态），SPP=1 → PPLV=0
- 提取 SPIE 位：SPIE=1 → 设置 PRMD.PIE
- 写入 PRMD，供 ertn 使用

6.2 hal/loongarch/hal_tlbrefill.S 逐行详解

这是整个 LoongArch 移植中最重要、最复杂、调试时间最长的代码。

6.2.1 入口和 PS 设置 (行 10-19)

```

1 tlb_refill_entry:
2     csrwr    $t0, 0x8b          # 将 $t0 保存到 TLBRSAVE
3                                     # 同时 $t0 = 旧的 TLBRSAVE (丢弃)
4
5     csrrd    $t0, 0x8e          # 读 TLBREHI
6     srli.d   $t0, $t0, 6        # 右移 6 位, 丢弃低 6 位 (原 PS 值)
7     slli.d   $t0, $t0, 6        # 左移 6 位, 低 6 位置 0
8     ori      $t0, $t0, 12       # PS = 12 (4KB 页大小)
9     csrwr    $t0, 0x8e          # 写回 TLBREHI

```

为什么需要设置 **PS**: TLBREHI.PS (Page Size) 字段是软件管理的。TLB 重填时硬件自动设置 TLBREHI.VPPN (从故障 VA), 但 PS 字段不会自动设置。如果不设置 PS=12, tlbfill 可能将条目填入 MTLB (大页表) 而非 STLB (标准页表), 导致后续 TLB 查找无法匹配 4KB 页。

srli.d + slli.d 组合的作用: 清除低 6 位 (原 PS 值), 保留 VPPN, 然后用 **ori** 设置新的 PS=12。

6.2.2 MAXVA 边界检查 (行 21-28)

```

1     csrrd    $t0, 0x89          # 读 TLBRBADV (故障 VA)
2     srli.d   $t0, $t0, 38       # 检查 bit 38+ (MAXVA = 1<<38)
3     bnez     $t0, .Ltlbr_invalid # 如果 bit 38+ 非零, VA 超出范围
4
5     csrrd    $t0, 0x89          # 重新读 TLBRBADV
6     srli.d   $t0, $t0, 13       # 右移 13 位
7     beqz     $t0, .Ltlbr_invalid # 如果移位后为 0, VA < 2*PGSIZE

```

两段检查分别保护什么:

- 第一段: **srli.d 38** 后检查是否非零。如果故障 VA 的 bit 38 或更高位被置位, 说明 $VA \geq 1 \ll 38 = \text{MAXVA}$ 。这是 xv6 用户地址空间的上限, 超出此范围的访问必须失败。
- 第二段: **srli.d 13** 后检查是否为零。如果移位后为 0, 说明 $VA < 2 * \text{PGSIZE} = 0x2000$ 。这保护了 **exec** 在低 2 页设置的 PLV0-only 保护页。如果用户态访问这些页 (例如 NULL 指针解引用), TLB handler 拒绝填充, 触发缺页异常, kernel 杀死进程。

6.2.3 4 级 lddir walk (行 29-56)

```

1     csrrd    $t0, 0x19          # 读 PGDL (页表根物理地址)
2
3     # Level 3: PGD, VA[47:39]
4     lddir    $t0, $t0, 3        # 硬件 walk: 从 $t0 指向的 PGD 页中
5                                     # 按 TLBRBADV[47:39] 索引读 PTE
6     andi     $t0, $t0, 1        # 检查 V 位 (bit 0)
7     beqz     $t0, .Ltlbr_invalid # V=0 → 页表项无效
8     csrrd    $t0, 0x19          # 重新读 PGDL
9     lddir    $t0, $t0, 3        # 重新 walk Level 3
10    addi.d   $t0, $t0, -1       # 清除 V 标志: {PPN,1} - 1 = PPN

```


lddir 指令的精确行为：

lddir rd, rj, level 执行以下操作：

1. 从 rj 获取当前级页表的物理基址
2. 从 TLBRBADV 获取故障 VA
3. 根据 PWCL/PWCH 配置，计算该级的索引位 (level=3 → VA[47:39])
4. 读 PTE = *(rj + index*8)
5. 如果 PTE.V=1：返回 {PTE.PPN[47:12], 1} (基址 + V 标志)
6. 如果 PTE.V=0：返回无效值

为什么先检查 V 再重走：第一次 lddir; andi; beqz 只用来检查 PTE 是否有效。如果有效，再从 PGDL 重新走一遍得到完整结果。这是因为 andi 破坏了 \$t0 中 lddir 的结果。

addi.d -1 为什么清除了 V 标志：lddir 成功时返回 {PPN_base, 1} (物理基址 OR 1)。物理基址是页对齐的 (低 12 位 = 0)，bit 0 = 0。所以 {PPN_base, 1} = PPN_base + 1。addi.d -1 将 PPN_base + 1 - 1 = PPN_base，得到纯净的下一级页表物理地址。

6.2.4 ldpte 和 P/W 位清除 (行 58-65)

```

1      ldpte    $t0, 0                # 读偶数 PTE → TLBRELO0
2      ldpte    $t0, 1                # 读奇数 PTE → TLBRELO1
3
4      csrrd     $t0, 0x8c             # 读 TLBRELO0
5      bstrins.d $t0, $r0, 8, 7        # 清除 bit[8:7] (P 和 W)
6      csrwr     $t0, 0x8c             # 写回 TLBRELO0
7      csrrd     $t0, 0x8d             # 读 TLBRELO1
8      bstrins.d $t0, $r0, 8, 7        # 同样清除
9      csrwr     $t0, 0x8d             # 写回 TLBRELO1

```

ldpte 的精确行为：

ldpte rj, seq 执行：

1. 从 rj 获取 PTE 页的物理基址
2. seq=0 读偶数 PTE (VA[12]=0)，seq=1 读奇数 PTE (VA[12]=1)
3. 将内存 PTE 格式转换为 TLBELO 格式并写入对应 CSR

为什么需要 bstrins.d 清除 bit7/8：

内存 PTE 格式 (手册 Figure 8) 中：

- bit 7 = P (硬件页表遍历器使用，表示“物理页存在”)
- bit 8 = W (硬件页表遍历器使用，表示“页可写”)

但 TLBELO 寄存器中，bit[8:7] 是保留位 (必须为 0)。ldpte 指令会自动处理 PTE→TLBELO 的格式转换，但不保证清除保留位。bstrins.d 显式将这两个 bit 清零，确保 TLB 条目不被污染。

bstrins.d 指令：bstrins.d rd, rj, msb, lsb 将 rj 的 bits[msb:lsb] 插入 rd 的对应位置。此处 rj=\$r0 (全零)，效果是清零 rd 的 bits[8:7]。

6.2.5 tlbfill 和 ertn (行 66-69)

```

1      tlbfill                    # 随机填入 TLB 槽
2      csrrd     $t0, 0x8b        # 从 TLBRSAVE 恢复 $t0

```

3	<code>ertn</code>	# 异常返回
---	-------------------	--------

`tlbfill` 的行为：

- 根据 TLBIDX.PS 和 STLBPS 选择一个随机 TLB 槽
- 将 TLBRELO0/1 + TLBREHI 的内容写入该槽
- 因为此时 IsTLBR=1，填入的条目自动设为有效 (V=1)

`ertn` 的行为：

- 从 TLBRPRMD 恢复 CRMD.PLV 和 CRMD.IE
- 从 TLBRERA 恢复 PC
- 设置 CRMD.DA=0, CRMD.PG=1
- 清除 IsTLBR 标志

6.3 hal/loongarch/hal_start.c 逐行详解

6.3.1 start() 函数（行 22-72）

```

1 void start() {
2     int id = r_cpuid();
3     w_tp(id);                      // tp = cpuid (关键! hal_get_hartid 从此读取)
4
5     if (id == 0) {
6         // === Core 0: 完整初始化 ===
7
8         // 早期 UART 输出 (未经 uartinit, 直接写 MMIO)
9         volatile char *uart_thr = (volatile char*)0x1FE001E0;
10        const char *msg = "start: begin\n";
11        while (*msg) { *uart_thr = *msg++; }
12
13        // 从 flash LMA 复制 .data 初始值到 RAM VMA
14        uint64 *src = (uint64*)_data_lma;
15        uint64 *dst = (uint64*)_data_start;
16        while(dst < (uint64*)_bss_start) *dst++ = *src++;
17
18        // 清零 RAM 中的 .bss
19        dst = (uint64*)_bss_start;
20        while(dst < (uint64*)_bss_end) *dst++ = 0;
21
22        // 内存屏障 + 设置就绪标志
23        __sync_synchronize();
24        *READY_FLAG = 1;
25    } else {
26        // === 其他核: 等待 Core 0 完成内存初始化 ===
27        while (*READY_FLAG == 0) ;
28        __sync_synchronize();
29    }
30
31    // 所有核: 设置异常入口
32    w_eentry((uint64)kernelvec);

```

```

33
34 // 使能 FPU
35 { uint64 euen = 1; asm volatile("csrwr %0, 0x15" : "+r"(euen)); }
36
37 main();
38 }

```

这段代码的每一行为什么这样写：

1. **w_tp(id)**: tp 寄存器是 hal_get_hartid() 的数据源。必须在这里设置，因为后续所有内核代码都通过 hal_get_hartid() → r_tp() 获取核心 ID
2. 直接 **UART MMIO** 写：此时 uartinit() 还没有被调用（它在 main()→consoleinit() 中）。用 volatile char* 直接写 UART THR 寄存器（0x1FE001E0）实现早期输出
3. **__data_lma** 到 **__data_start** 的复制：链接脚本将 .data 的 LMA（加载地址）放在 flash 中，VMA（虚拟地址）放在 RAM 中。内核启动时必须手动将初始值从 flash 搬运到 RAM。
__data_lma, __data_start, _bss_start, _bss_end 都是链接器导出的符号
4. **___sync_synchronize()**: 内存屏障，确保所有写操作对其他核心可见
5. **READY_FLAG**: 固定地址 0x08100000（RAM 中，kalloc 范围外）。Core 0 写入 1 表示内存初始化完成，其他核自旋等待
6. **w_entry((uint64)kernelvec)**: 设置通用异常入口。kernelvec 在 hal_kvec.S 中定义。TLB refill 入口在 kvmminihart() 中单独设置
7. **FPU 使能**: LoongArch 复位后 FPU 默认禁用。EUEEN CSR (0x15) bit 0 控制 FPU 使能。必须在内核使用浮点操作前（例如 printf 的浮点格式）使能

6.3.2 timerinit() 函数（行 74-87）

```

1 void timerinit() {
2     // 在 ECFG 中使能定时器中断 (LIE bit 11)
3     w_ecfg(r_ecfg() | ECFG_LIE_TIMER);
4
5     // 清除挂起的中断
6     w_ticlr(1);
7
8     // 配置定时器：周期模式，~10ms 间隔
9     // QEMU la464 StableCounter 约 100MHz
10    uint64 interval = 1000000;
11    w_tcfg((interval << TCFG_INITVAL_SHIFT) | TCFG_PERIOD | TCFG_EN);
12 }

```

TCFG 寄存器格式：

- bit 0 = EN（定时器使能）
- bit 1 = PERIOD（周期模式。TVAL 减到 0 后自动重载为 INITVAL）
- bits[31:2] = INITVAL（初始倒计时值）

1000000 个周期 / 100MHz = 10ms。这是 xv6 标准的调度时间片。

6.4 hal/loongarch/hal_virtio.c 详解

6.4.1 RAM-disk 数据结构

```

1 struct ramdisk {
2     uchar *pages[RAMDISK_MAX_PAGES]; // 指向 kalloc 分配的物理页
3     uint npages;                      // 总页数
4     uint size;                        // 总字节数
5     struct spinlock lock;             // 保护并发访问
6 };
7 static struct ramdisk ramdisk[NRAMDISK]; // 最多 2 个盘

```

每个 ramdisk 是一个页指针数组。文件系统镜像被切分为 4KB 页，每页用 kalloc 分配独立的物理页并复制数据。

6.4.2 ramdisk_init() 的 block 映射

```

1 static void ramdisk_init(int idx, uchar *img_start, uchar *img_end) {
2     struct ramdisk *rd = &ramdisk[idx];
3     uint size = (uint)(img_end - img_start);
4     rd->size = size;
5     rd->npages = (size + PGSIZE - 1) / PGSIZE;
6
7     for (uint i = 0; i < rd->npages; i++) {
8         rd->pages[i] = kalloc(); // 分配可写物理页
9         uint n = PGSIZE;
10        if (i == rd->npages - 1)
11            n = size - i * PGSIZE; // 最后一页可能不满 4KB
12        memmove(rd->pages[i], img_start + i * PGSIZE, n);
13    }
14 }

```

为什么需要 kalloc 复制：img_start 指向的是内核.rodata 段中嵌入的 fs.img 镜像。这个地址在 flash 中（只读）。文件系统写操作需要可写内存，因此必须为每页分配 RAM 页并复制镜像数据。

6.4.3 virtio_disk_rw() 的同步 I/O

```

1 void virtio_disk_rw(struct buf *b, int write) {
2     struct ramdisk *rd = ramdisk_for_dev(b->dev);
3     uint off = b->blockno * BSIZE; // 字节偏移
4
5     for (uint i = 0; i < BSIZE; i += PGSIZE) {
6         uint pg = (off + i) / PGSIZE; // 页号
7         uint po = (off + i) % PGSIZE; // 页内偏移
8         uint n = PGSIZE - po;
9         if (n > BSIZE - i) n = BSIZE - i;
10
11        if (write)
12            memmove(rd->pages[pg] + po, b->data + i, n);

```

```

13     else
14         memmove(b->data + i, rd->pages[pg] + po, n);
15     }
16     b->disk = 0; // 同步完成：直接标记为完成
17     wakeup(b);  // 唤醒等待者
18 }

```

与 RISC-V virtio 驱动的关键区别：

- RISC-V 版本需要构造 virtqueue 描述符、写 MMIO 寄存器、等待中断
- RAM-disk 版本直接 memmove + 标记完成，无需任何硬件交互
- 同步完成意味着 b->disk = 0 在函数返回前就设置好了
- wakeup(b) 唤醒任何在 sleep(b, ...) 上等待的进程

7 HAL 接口头文件详解

7.1 hal/hal.h ——统一入口

Listing 1: hal.h 核心逻辑

```

1  #ifdef ARCH_riscv
2  #include "arch.h"
3  #include "memlayout.h"
4  #elif defined(ARCH_loongarch)
5  #include "arch.h"
6  #include "memlayout.h"
7  #else
8  #error "ARCH_xxx not defined"
9  #endif
10
11 #include "hal_arch.h"
12 #include "hal_vm.h"
13 #include "hal_intr.h"
14 #include "hal_timer.h"
15 #include "hal_memlayout.h"
16 #include "hal_console.h"
17 #include "hal_ctx.h"

```

设计原理：内核代码只 #include "hal/hal.h"，不需要知道当前架构。arch.h 和 memlayout.h 通过 Makefile 的 -Ihal/\$(ARCH) 路径解析到不同文件。#error 指令确保编译时检查 ARCH 符号必须定义。

7.2 hal/hal_arch.h ——CPU 控制接口

声明了 14 个 HAL 函数，将 RISC-V 的 CSR 操作抽象为统一接口。最关键的三个：

- hal_read_scause()：LoongArch 上调用 estat_to_scause() 转换异常码，使得 trap.c 可以继续用 RISC-V 的 scause 值（8=syscall, 13=load fault, 15=store fault）做判断
- hal_read_sstatus() / hal_write_sstatus()：LoongArch 上通过 r_sstatus_compat() 将 PRMD 转换为 RISC-V sstatus 格式

- `hal_get_hartid()`: RISC-V 读 `tp` 寄存器 (规避 `mhartid` 的 M 态限制), LoongArch 读 `CPUID CSR`

7.3 hal/hal_ctx.h —— 上下文结构体

Listing 2: ARCH 条件结构体定义

```

1  #ifdef ARCH_loongarch
2  // 12 registers: ra, sp, fp, s0-s8
3  struct hal_context {
4      uint64 ra, sp, fp, s0, s1, s2, s3, s4, s5, s6, s7, s8;
5  };
6  #else
7  // 14 registers: ra, sp, s0-s11 (RISC-V)
8  struct hal_context {
9      uint64 ra, sp, s0, s1, s2, s3, s4, s5, s6, s7, s8, s9, s10, s11;
10 };
11 #endif
12 void hal_switch(struct hal_context *old, struct hal_context *new);

```

为什么 **LoongArch** 少 2 个寄存器: LoongArch ABI 将 `$r22` 指定为帧指针 (`fp`), 而 RISC-V 的 `fp` 是 `s0` 的别名。所以 LoongArch 的被调用者保存寄存器是 `ra, sp, fp, s0-s8` (12 个), RISC-V 是 `ra, sp, s0-s11` (14 个)。结构体布局必须与 `hal_swch.S` 中的汇编 `offset` 精确匹配。

7.4 hal/hal_intr.h —— 中断控制器接口

声明 4 个函数: `hal_irq_init()`, `hal_irq_hart_init()`, `hal_irq_claim()`, `hal_irq_complete()`。这个接口反映了 PLIC 的 `claim/complete` 模型, 同样适用于 EIOINTC。

7.5 hal/hal_timer.h —— 定时器接口

```

1  uint64 hal_get_time(void);           // 读当前时间
2  void hal_set_timer(uint64 next);    // 设下次中断
3  void hal_timer_init(void);          // 初始化定时器

```

RISC-V 实现用 `r_time() + w_stimecmp()` (`Sstc` 扩展), LoongArch 实现用 `rdtime.d + TCFG/TVAL/TICLR CSR` 系列。

7.6 hal/hal_console.h —— 控制台 I/O 接口

```

1  void hal_console_init(void);
2  void hal_putchar_sync(int c);       // 同步阻塞输出 (panic 安全)
3  void hal_putchar(int c);            // 中断驱动输出
4  int hal_getchar(void);              // 读输入字符
5  void hal_console_intr(void (*handler)(int)); // 注册中断回调

```

双输出路径 (同步 vs 中断驱动) 允许 `printf` 在 `panic` 期间使用同步轮询输出, 而正常 `write()` 系统调用使用中断驱动方式释放 CPU。

8 RISC-V HAL 实现详解

8.1 hal/riscv/arch.h ——PTE 与页表宏 (3 级 Sv39)

PTE 格式：RISC-V Sv39 页表，每个 PTE 64 位，PPN 在 [53:10] 连续存储：

```
1 #define PTE_V (1L << 0) // Valid
2 #define PTE_R (1L << 1) // Readable
3 #define PTE_W (1L << 2) // Writable
4 #define PTE_X (1L << 3) // Executable
5 #define PTE_U (1L << 4) // User-accessible
6
7 #define PA2PTE(pa) (((uint64_t)pa) >> 12) << 10)
8 #define PTE2PA(pte) (((pte) >> 10) << 12)
9 #define PTE_FLAGS(pte) ((pte) & 0x3FF)
```

页表索引：3 级 9-9-9-12 寻址：

```
1 #define PT_LEVELS 3
2 #define PXSHIFT(level) (12 + 9*(level))
3 #define PX(level, va) (((uint64_t)(va) >> PXSHIFT(level)) & 0x1FF)
4 #define MAXVA (1L << 38) // bit 38 是符号扩展位
```

HAL 统一包装器 (第 343-362 行) 将 19 个 RISC-V CSR 函数映射为 hal_ 前缀：

```
1 static inline uint64_t hal_get_hartid(void) { return r_tp(); }
2 static inline uint64_t hal_read_scause(void) { return r_scause(); }
3 static inline uint64_t hal_read_sepc(void) { return r_sepc(); }
4 static inline void hal_write_sepc(uint64_t x) { w_sepc(x); }
5 // ... 共 19 个
```

关键技巧——tp 寄存器传递 hartid：hal_get_hartid() 在 RISC-V 上调用 r_tp() 而非 r_mhartid()。因为 mhartid 是 M 态 CSR，start() 执行 mret 切换到 S 态后无法访问。start() 在 M 态时将 hartid 写入 tp 寄存器，S 态通过 tp 读取。tp 寄存器在 RISC-V 特权架构中跨模式可用。

8.2 hal/riscv/hal_start.c ——M 态初始化

start() 在 M 态执行，配置以下内容后通过 mret 跳转到 S 态的 main()：

1. mstatus.MPP = S_MODE ——mret 后进入 S 态
2. mepc = main ——mret 跳转到 main()
3. medeleg = 0xffff, mideleg = 0xffff ——所有异常/中断委托给 S 态
4. pmpcfg0 = 0xf, pmpaddr0 = 全地址空间 ——解锁所有物理内存
5. timerinit() ——配置定时器首次中断
6. w_tp(mhartid) ——将 hartid 存入 tp (关键!)
7. mret

PMP 解锁是必须的：RISC-V 复位后 PMP 默认拒绝所有访问。pmpcfg0 = 0xf 设置 NAPOT 模式（自然对齐的 2 的幂大小区域），pmpaddr0 覆盖全部 56 位物理地址，权限为 RWX。

8.3 hal/riscv/hal_tramp.S ——用户态陷入/返回

uservec (用户 → 内核):

1. `csrw sscratch, a0` ——保存用户 `a0` 到 `sscratch` CSR
2. `csrr a0, sscratch` ——`a0` = `TRAPFRAME` 地址
3. 保存全部 31 个用户寄存器到 `trapframe` (offset 40-280)
4. 从 `sscratch` 恢复用户 `a0`, 存入 `trapframe` offset 112
5. 加载内核 `sp`、`hartid`、`usertrap` 地址、内核 `satp`
6. `sfence.vma; cswr satp, t1; sfence.vma` ——切换内核页表
7. `jalr t0` ——跳转 `usertrap()`

userret (内核 → 用户):

1. `sfence.vma; cswr satp, a0; sfence.vma` ——切换用户页表
2. 从 `trapframe` 恢复全部 31 个用户寄存器
3. `sret` ——返回用户态

sscratch 技巧: `uservec` 不能直接用 `a0` 访问 `trapframe` (`a0` 是用户数据), 所以先 `cswr sscratch, a0` 将用户 `a0` 暂存到 `sscratch` CSR, 再用 `a0` 加载 `trapframe` 地址。最后从 `sscratch` 恢复用户 `a0` 写入 `trapframe`。

8.4 hal/riscv/hal_virtio.c ——Virtio-blk MMIO 驱动

373 行的完整 `virtio-blk` 驱动, 实现标准 `virtio` 拆分 `virtqueue` 协议:

1. 设备发现: 验证 Magic Value (`0x74726976` = "virt") 和 Device ID (`2` = block)
2. 功能协商: 禁用不需要的特性 (`VIRTIO_BLK_F_RO`, `F_SCSI` 等)
3. 队列初始化: `kalloc` 分配 `desc/avail/used` 三个内存区域
4. I/O 流程: 构造 3 个描述符链 (请求头 + 数据 + 状态字节)
5. 中断处理: 读取 `used ring`, `wakeup` 等待的进程

三描述符链是 `virtio-blk` 规范 §5.2 的要求:

- Desc 0: 请求头 (type + reserved + sector) ——设备读取
- Desc 1: 数据缓冲区 ——设备读写
- Desc 2: 状态字节 ——设备写入 (0= 成功)

9 LoongArch HAL 实现详解

9.1 hal/loongarch/arch.h ——PTE 格式与 RISC-V 兼容层

9.1.1 PTE 格式 (4 级页表)

LoongArch 的 PTE 格式与 RISC-V 完全不同。手册 Figure 8 定义:

```
1 // PTE bit layout (LoongArch Manual 5.4.5, Figure 8)
2 // [63:60] VSEG, [59:12] PPN[47:0], [11:7] PPN[52:48],
3 // [6] G, [5:4] MAT, [3:2] PLV, [1] D, [0] V
4
5 #define PTE_V      (1L << 0)    // Valid
6 #define PTE_D      (1L << 1)    // Dirty (writable)
```



```

7 #define PTE_PLV3 (3L << 2) // User-mode (PLV=3)
8 #define PTE_MAT (1L << 4) // MAT[0]=1: Cacheable Coherent
9 #define PTE_HW_P (1L << 7) // Hw walker present flag
10 #define PTE_HW_W (1L << 8) // Hw walker writable flag
11
12 #define PTE_W (PTE_D | PTE_HW_W)
13 #define PTE_U PTE_PLV3
14 #define PTE_R (0) // default readable when NR=0
15 #define PTE_X (0) // default executable when NX=0
16
17 #define PTE_V_CACHE (PTE_V | PTE_MAT | PTE_HW_P | PTE_HW_W)

```

PTE_V_CACHE 的设计意图：LoongArch 的叶子 PTE 必须设置 MAT 位来表示缓存属性。MAT=01 (CC) 表示一致性可缓存 (Coherent Cached)，是普通内存页的标准属性。PTE_HW_P 和 PTE_HW_W 是硬件页表遍历器使用的标志，在 TLBELO 中是保留位，因此 TLB refill handler 需要清除它们。

9.1.2 PA2PTE/PTE2PA ——单掩码简化

由于本项目的物理地址限制在 48 位以内 (QEMU virt 的 RAM 不超 256MB)，PPN 的高位 (PTE[11:7]) 始终为零。因此可以用单掩码简化：

```

1 #define PA2PTE(pa) ((uint64)(pa) & 0x0FFFFFFFFF000ULL)
2 #define PTE2PA(pte) ((uint64)(pte) & 0x0FFFFFFFFF000ULL)

```

这比标准的分段 PPN 格式简单得多，因为不需要分别处理 PTE[59:12] 和 PTE[11:7]。

9.1.3 4 级页表索引

```

1 #define PT_LEVELS 4
2 #define PXSHIFT(level) (12 + 9*(level))
3 #define PX(level, va) (((uint64)(va) >> PXSHIFT(level)) & 0x1FF)
4 #define MAXVA (1ULL << 38)

```

与 RISC-V 的关键差异：

- PT_LEVELS = 4 (RISC-V = 3)：多一级 PGD (VA[47:39])
- MAXVA = 1ULL<<38 (与 xv6 Sv39 一致)：虽然 LoongArch 支持 48-bit VA，但为了保持 xv6 原始地址契约

9.1.4 estat_to_scause() ——异常码转换

这是 LoongArch HAL 中最关键的兼容层函数。LoongArch 的 ESTAT.Ecode 与 RISC-V scause 的值完全不同。例如 LoongArch 的 ADE (Ecode=8) 恰好与 RISC-V 的 syscall (scause=8) 冲突。因此必须做显式映射，不能透传：

```

1 static inline uint64 estat_to_scause(uint64 estat) {
2     uint64 ecode = (estat >> 16) & 0x3F;
3     uint64 esubcode = (estat >> 22) & 0x1FF;
4

```

```

5  if (ecode == 0) {                                // Interrupt
6      if (estat & (1<<11)) return 0x8000000000000005ULL; // timer
7      if (estat & (1<<12)) return 0x8000000000000009ULL; // external
8      return 0x8000000000000000ULL | ecode;
9  }
10
11  switch (ecode) {
12      case 11: return 8;    // SYS → RISC-V syscall
13      case 1: return 13;    // PIL → load page fault
14      case 2: return 15;    // PIS → store page fault
15      case 3: return 12;    // PIF → instr page fault
16      case 4: return 15;    // PME → store fault
17      case 8: return esubcode==0 ? 12 : 13; // ADE → avoid syscall conflict!
18      case 13: return 2;    // INE → illegal instruction
19      case 14: return 12;   // IPE → instr page fault
20      default: return 0x100 | ecode; // unknown: keep unique
21  }
22 }

```

为什么不能用 **default: return ecode**: 因为 LoongArch 的 ecode 8 (ADE) 与 RISC-V 的 ecode 8 (syscall) 冲突。如果直接返回 ecode，内核会把地址错误当成系统调用来处理，导致不可预测的行为。0x100 | ecode 确保未知码有唯一值。

9.1.5 RISC-V 兼容层

```

1  #define w_satp(x)    w_pgdl(x)
2  #define r_satp()    r_pgdl()
3  #define w_stvec(x)  w_eentry(x)
4  #define r_stvec()   r_eentry()
5  #define sfence_vma() asm volatile("invtlb 0, $r0, $r0")

```

这些 #define 别名让 kernel 通用代码中的 RISC-V CSR 调用在 LoongArch 上自动转换为对应的 LoongArch CSR。例如 vm.c 中的 w_satp(MAKE_SATP(kpgtbl)) 在 LoongArch 编译时等同于 w_pgdl(kpgtbl)。

SSTATUS 兼容: r_sstatus_compat() 从 PRMD 重建 RISC-V sstatus 格式:

- PRMD.PPLV != 3 (来自内核态) → SSTATUS_SPP = 1
- PRMD.PIE = 1 → SSTATUS_SPIE = 1

9.2 hal/loongarch/hal_tlbrefill.S —— TLB 重填处理器

这是整个 LoongArch 移植中最核心的文件。

LoongArch MMU 在 TLB miss 时不像 RISC-V 那样自动走页表。而是触发 TLB 重填异常 (TLBR)，跳转到 TLBENTRY (CSR 0x88) 指向的 handler。

Handler 执行环境 (关键约束):

- DA=1, PG=0 —— 直接地址翻译模式, VA=PA
- 不能使用内核栈 (\$sp 指向用户态 VA, 在 DA=1 下 PA 可能无 RAM)
- 只能用 TLBSAVE CSR 保存 1 个临时寄存器

- 只能访问物理地址空间的页表页

完整流程：

1. `csrwr $t0, 0x8b` —— 保存 `$t0` 到 `TLBRSAVE`
2. 设置 `TLBREH.LPS = 12` (4KB 页大小)
3. **MAXVA** 边界检查：
 - 读 `TLBRBADV`，检查 bit 38+ 是否置位 ($VA \geq 1 \ll 38$)
 - 检查 $VA < 2 * PGSIZE$ (低于保护页)
 - 任一条件成立则跳 `.Ltlbr_invalid`
4. 读 `PGDL` 获取页表根物理地址
5. 4 级 **lddir walk**：
 - `lddir level=3 (VA[47:39])` → 检查 `V` → `addi.d -1` 清 `V` 标志 → `PUD` 基址
 - `lddir level=2 (VA[38:30])` → 检查 `V` → `addi.d -1` → `PMD` 基址
 - `lddir level=1 (VA[29:21])` → 检查 `V` → `addi.d -1` → `PTE` 基址
6. `ldpte seq=0/1` → 将偶数/奇数 `PTE` 写入 `TLBRELO0/1`
7. 清除 **P/W** 保留位：`bstrins.d $t0, $r0, 8, 7` 清零 bit7/8
8. `tlbfill` → 填入随机 TLB 槽
9. `ertn` → 返回，`CRMD.PLV ← TLBRPRMD.PPLV`

为什么每级都要从 **PGDL** 重走：因为 handler 只剩一个临时寄存器 (`$t0`)，无法同时保存 `PGD/PUD/PMD` 的中间基址。另一种方案是用 `KSave CSR` 保存，但此处选择了“从根重走”的简单方案。

为什么 `addi.d -1` 清除 **V** 标志是正确的：`lddir` 的结果格式是 $\{PPN_{base}, V=1\}$ 。`V` 标志位于 bit 0。物理页基址页对齐，低 12 位为 0，bit 0 = 0。`addi.d -1` 从 $\{PPN_{base}, 1\}$ 减去 1，得到 PPN_{base} (因为 bit 0 从 1 变 0，无借位)。这是 LoongArch 软件 TLB walk 的标准技巧。

为什么 `.Ltlbr_invalid` 要填无效条目：如果 handler 直接 `ertn` 而不填 TLB，CPU 重试访问时再次 TLB miss → 再次进入 handler → 无限循环。填一个 `V=0` 的条目后，重试时会 TLB hit (`V=0`) → 触发标准缺页异常 (`PIL/PIS/PIF`) → 进入 `usertrap` → `vmfault` 处理。

9.3 hal/loongarch/hal_tramp.S —— `uservec/userret`

与 RISC-V 版本最大的区别是 **KSave0/KSave1 CSR** 替代 `sscratch`：

- **KSave1 (CSR 0x31)**：永久保存当前进程 `trapframe` 的内核虚拟地址。由于 `DMW0` 恒等映射所有低物理地址，此 `KVA` 在 `DA=0, PG=1` 模式下通过 `DMW0` 直接可达。
- **KSave0 (CSR 0x30)**：在 `uservec` 入口暂存用户 `a0`

`uservec` 流程：

1. `csrwr $a0, 0x30` —— `KSave0 = 用户 a0`
2. `csrrd $a0, 0x31` —— `a0 = trapframe KVA`
3. 保存全部 31 个 GPR 到 `trapframe` (通过 `DMW0` 直接访问物理页)
4. 从 `KSave0` 恢复用户 `a0`，存入 `trapframe offset 112`
5. 加载内核 `sp`、`tp` (`hartid`)、`usertrap` 地址、内核 `PGDL`
6. 切换 `PGDL` 到内核页表 (无需 `invtlb`，内核走 `DMW0`)
7. `jirl $ra, $t0, 0` —— 跳转 `usertrap()`

与 RISC-V 的关键差异：

- 不需要将 trapframe 映射到用户页表的 TRAPFRAME VA ——KSave1 直接持有物理 KVA
- 不需要 sscratch 技巧——KSave0/1 是独立的 CSR，不会与用户 a0 冲突
- 不需要在用户页表映射 trampoline ——trampoline 在 flash 0x1C009000，DMW0 可达
- 切换内核 PGDL 后不需要 invtlb ——内核全部走 DMW0，不依赖 TLB

9.4 hal/loongarch/hal_start.c ——flash→RAM 搬运

这是 LoongArch 独有的设计。QEMU -bios 将整个内核二进制加载到只读 flash(0x1C000000)，但.data 和.bss 必须位于可写 RAM。链接脚本通过 AT() 将.data 的初始值放在 flash，VMA 设在 RAM (0x00400000)。start() 负责搬运：

```
1 // Core 0 only:
2 uint64 *src = (uint64*)_data_lma; // flash 中的初始值
3 uint64 *dst = (uint64*)_data_start; // RAM VMA
4 while(dst < (uint64*)_bss_start) *dst++ = *src++;
5 // 清零 .bss
6 dst = (uint64*)_bss_start;
7 while(dst < (uint64*)_bss_end) *dst++ = 0;
```

多核同步：Core 0 完成搬运后设置 boot_done = 0x6c613634646f6e65 ("la64done")，其他核自旋等待此标记。

9.5 hal/loongarch/hal_virtio.c ——RAM-disk 块设备

LoongArch QEMU virt 的 virtio 是 PCI 设备（非 MMIO），需要完整的 PCI 枚举和 virtio-over-PCI 传输层。为避免这部分复杂度，采用 RAM-disk 替代方案：

1. 构建时 objcopy -I binary -O elf64-loongarch fs.img kernel/fs_img.o
2. 链接时将 fs_img.o 嵌入内核，产生 _binary_fs_img_start/end 符号
3. 启动时 ramdisk_init() 用 kalloc 分配可写页，从嵌入镜像复制数据
4. I/O 时直接从 RAM 页读写，同步完成 (b->disk = 0)

双盘支持：VIRTIO_NDISK=2 时链接 fs1_img.o，dev=2 对应第二块盘。

10 内核修改文件详解

10.1 kernel/vm.c ——页表管理

10.1.1 kvmmake() ——LoongArch 的特殊映射

LoongArch 的内核物理内存布局是 flash (0x1C000000) + RAM (0x00400000)，与 RISC-V 的连续 RAM (0x80000000+) 完全不同。kvmmake() 的 LoongArch 分支需要：

- 分开映射 flash 代码段和 RAM 数据段
- TRAMPOLINE 在 flash 中 (0x1C009000)，跳过映射它周围的代码
- PLIC 范围 (0x0FE00000-0x13E00000) 覆盖 VIRTIO0，单独映射

10.1.2 kvmminithart() ——MMU 使能

LoongArch 的 MMU 初始化远比 RISC-V 复杂：

1. 设 DMW0: VSEG=0, PLV0=1, MAT=CC
2. 设 PWCL/PWCH: 4 级 9-9-9-12 配置
3. 设 RVACFG=8: 降低硬件有效 VA 位宽
4. 设 TLBRENTY: 指向 tlb_refill_entry
5. 清除 CRMD.DA, 设置 CRMD.PG: 启用映射地址翻译

10.1.3 walk() ——跨架构页表遍历

```

1 for(int level = PT_LEVELS - 1; level > 0; level--) {
2     pte_t *pte = &pagetable[PX(level, va)];
3     if(*pte & PTE_V) {
4         pagetable = (pagetable_t)PTE2PA(*pte);
5     } else {
6         if(!alloc || (pagetable = kalloc()) == 0) return 0;
7         memset(pagetable, 0, PGSIZE);
8         *pte = PA2PTE(pagetable) | PTE_V;
9     }
10 }
11 return &pagetable[PX(0, va)];

```

通过 PT_LEVELS (RISC-V=3, LoongArch=4) 和 PXSHIFT/PX 宏 (两个 arch.h 各自定义), 同一个 walk() 函数适配两种架构。

10.1.4 freewalk() ——叶子/非叶子区分

两个架构用不同的方法区分叶子和非叶子 PTE：

- RISC-V: 叶 PTE 有 R/W/X 权限位, 非叶只有 V
- LoongArch: 叶 PTE 有 MAT 位 (bit 4), 非叶没有 (页对齐的 PA 保证 bit 4=0)

10.1.5 vmfault() ——惰性分配 + LoongArch TLB 填充

LoongArch 版本在 vmfault 中增加了 tlb_fill_from_pte() 调用：

1. 如果页已映射 (PTE V=1) 但触发了缺页：说明 TLB 条目陈旧/无效，手工填 TLB
2. 如果页未映射 (PTE V=0)：kalloc + mappages + 手工填 TLB

10.2 kernel/trap.c ——陷阱处理

HAL 迁移程度：几乎完全使用 hal_* 函数，只有 2 处直接 CSR 访问：

- 第 41 行：r_sstatus() & SSTATUS_SPP ——快速检查陷阱来源
- 第 46 行：w_stvec((uint64)kernelvec) ——切换陷阱向量

LoongArch 特殊处理 (第 119-125 行)：

```

1 #ifdef ARCH_loongarch
2     uint64 tf_kva = (uint64)p->trapframe;
3     asm volatile("csrwr %0, 0x31" : "+r"(tf_kva)); // KSave1

```

```
4 #endif
```

在 `prepare_return` 中, 将当前进程 `trapframe` 的内核 VA 写入 `KSave1.trampoline(uservec)` 通过 `csrrd $a0, 0x31` 获取此地址。

10.3 kernel/exec.c ——ELF 加载器

LoongArch 独有: 在 `exec` 时预映射低 2 页为非用户保护页 (`PTE_U=0`):

```
1 #ifdef ARCH_loongarch
2     char *guard = kalloc();
3     memset(guard, 0, PGSIZE);
4     mappages(pagetable, 0, PGSIZE, (uint64)guard, PTE_R);
5     mappages(pagetable, PGSIZE, PGSIZE, (uint64)guard, PTE_R);
6 #endif
```

原因: QEMU la464 在 TLB 查找时忽略 VALEN 以上的 VA 位。像 $1 \ll 48$ 这样的地址可能别名到 VPPN 0。保护页使得此类别名触发陷阱 (`PTE_U=0` 拒绝用户访问) 而非命中用户数据。

因此用户程序从 `0x2000` 链接运行 (见 `user.ld`)。

10.4 kernel/proc.c ——进程管理

`proc_pagetable()` 中的 ARCH 条件: RISC-V 需要将 TRAMPOLINE 和 TRAPFRAME 映射到用户页表顶部, 但 LoongArch 不需要——因为 `trampoline` 通过 `DMW0` 访问, `trapframe` 通过 `KSave1` 访问。省略这些映射简化了用户地址空间管理。

11 设计思想总结

11.1 核心设计原则

1. 最小化内核修改: 7 个文件需要 ARCH 条件编译, 20+ 个文件完全平台无关
2. 翻译而非重写: `estat_to_scause()` 和 `sstatus_compat()` 让 LoongArch “伪装” 成 RISC-V
3. 硬件特性善用: LoongArch `DMW0` 直通内核空间, 避免内核 TLB miss
4. 简化复杂度: RAM-disk 替代 PCI virtio, 消除 PCI 枚举的复杂性
5. 保持 **xv6** 契约: $\text{MAXVA} = 1 \ll 38$, TRAPFRAME 语义, 低页保护

11.2 关键架构差异的处理策略

表 5: 架构差异的处理策略

差异	RISC-V	LoongArch
页表级数	Sv39 (3 级)	LA64 标准 (4 级)
适配方式	PT_LEVELS=3, walk 从 level=2 起	PT_LEVELS=4, walk 从 level=3 起
TLB miss	硬件自动 walk 页表	软件 TLB 重填
适配方式	无需 handler	hal_tlbrefill.S: lddir/ldpte 4 级 walk
异常返回	sret/mret	ertn (统一)
适配方式	hal_tramp.S 用 sret	hal_tramp.S 用 ertn
中断控制器	PLIC (MMIO)	EIOINTC (MMIO)
适配方式	hal_plic.c	hal_intr.c
定时器	CLINT+stimecmp	TCFG/TVAL CSR
适配方式	hal_timer.h 接口统一, 各平台实现内部不同	
virtio 磁盘	MMIO @0x10001000	PCI (需枚举)
适配方式	原生 MMIO 驱动	RAM-disk 替代
Callee-saved	14 寄存器	12 寄存器
适配方式	hal_ctx.h 条件编译, hal_swch.S 各写一份	
物理内存布局	连续 RAM (0x80000000+)	flash(0x1C000000) + RAM(0x00400000)
适配方式	kvmmake 分支, hal_start.c 搬运.data	

12 优化思路

12.1 从”代码尽量少”的角度

1. 统一 **hal_swch.S**: LoongArch 和 RISC-V 的 swch 逻辑相同, 差异只在寄存器数量和 offset。可以写一个脚本或宏生成两个版本, 减少手工维护两份汇编代码。
2. 合并 **hal_tramp.S** 的共同逻辑: uservec/userret 在两个架构上的差异主要在于 sscratch vs KSave0/1 和 sret vs ertn。其余寄存器保存/恢复的 offset 相同, 可以用条件汇编宏减少重复代码 (当前两份文件共 140+150=290 行)。
3. 消除 **console.c** 中的遗留直接调用: 第 71 行的 **uartwrite()** 直接调用可以通过 **hal_putchar()** 替代, 消除最后一个非 HAL 的 UART 调用。
4. 将 **tlb_fill_from_pte()** 移到 **hal_tlbrefill.S**: 当前在 **vm.c** 中用 C 内联汇编实现, 放在 **hal_tlbrefill.S** 中作为辅助函数更符合 HAL 分层原则, 且减少 **vm.c** 中的 ARCH 条件代码。
5. 统一 **PTE_V_CACHE** 语义: 当前 RISC-V 的 PTE_V_CACHE 退化为 PTE_V。可以在 RISC-V 侧也定义为一个表达式, 让 **mappages** 中无需平台判断。

12.2 从”系统运行稳定”的角度

1. **TLB shutdown**: 当前单核 (CPUS=1) 稳定, 多核 (CPUS>1) 未测试。LoongArch 的 `invtlb` 需要配合 IPI 实现跨核 TLB 失效。RISC-V 的 `sfence.vma` 已经处理了。
2. **TLB handler** 的原子性: 当前 handler 每级从 PGDL 重走, 如果两次 `lddir` 之间页表被修改 (另一个核或 DMA), 可能产生不一致。多核环境下需要加锁或使用原子操作。
3. **RVACFG** 的硬编码值: 当前 `w_rvacfg(8)` 是经验值, 应该通过 `CPUCFG` 指令读取实际 VALEN 后再设置。
4. **UART** 中断接收: 当前 UART 只有 TX (输出), 没有实现 RX 中断接收。虽然不影响 `usertests`, 但 shell 交互需要。
5. **RAM-disk** 写回: 当前 RAM-disk 的修改不会持久化回 `fs.img`。如果需要支持文件系统写操作持久化, 需要实现回写机制。
6. 异常码转换的完整性: `estat_to_scause` 的 `default` 分支返回 `0x100|ecode`。应该覆盖所有 LoongArch 定义的异常码 (当前只覆盖了 8 种)。缺少的包括: `FPDIS(15)`, `FPE(18)` 等。
7. **DMW** 覆盖范围验证: `DMW0` 覆盖 `VA[63:60]=0` 的所有地址。应该添加启动时检查, 确保 `KSTACK`、`UART`、`EIOINTC`、`TRAMPOLINE` 都在此范围内。
8. **virtio** 中断竞争: RISC-V 的 `virtio_disk_intr` 有锁保护, LoongArch 的 RAM-disk 是同步的无需锁。但如果将来替换为真实 `virtio`, 需要添加中断同步。